

数值算法 Homework 04

姓名: 雍崔扬

学号: 21307140051

Due: 08:00 Mar. 17, 2025

Problem 1

Interpolate the Runge function $f(x) = (1 + 25x^2)^{-1}$ over $[-1, 1]$

- ① using polynomials with a few equispaced nodes
- ② using polynomials with a few Chebyshev nodes
- ③ using Hermite spline with a few equispaced nodes

Visualize the results and estimate the interpolation error.

(1) 等距节点的 Newton 插值

设 $x_0 < x_1 < \dots < x_n$

因此可记 $x_i = x_0 + ih$ ($i = 0, \dots, n$) 或 $x_i = x_n - (n - i)h$

利用差商和差分的关系, 我们可以将 Newton 插值公式重写为前向插值公式:

设 $x = x_0 + ph$, 于是 $x - x_i = (p - i)h$

$$\begin{aligned} P(x) &= f(x_0) + f[x_0, x_1](x - x_0) + \dots + f[x_0, x_1, \dots, x_k](x - x_0)(x - x_1) \dots (x - x_{k-1}) \\ &\quad (\text{note that } f[x_0, x_1, \dots, x_k] = \frac{\Delta^k f_0}{h^k k!}) \\ &= f_0 + \frac{\Delta f_0}{h} \cdot ph + \frac{\Delta^2 f_0}{h^2 2!} \cdot ph \cdot (p - 1)h + \dots + \frac{\Delta^k f_0}{h^k} \cdot ph \cdot (p - 1)h \dots (p - k + 1)h \\ &= \binom{p}{0} f_0 + \binom{p}{1} \Delta f_0 + \binom{p}{2} \Delta^2 f_0 + \dots + \binom{p}{k} \Delta^k f_0 \\ &= \sum_{i=0}^k \binom{p}{i} \Delta^i f_0 \\ &\quad \text{where } \binom{p}{i} := \frac{p(p-1) \dots (p-i+1)}{i!} \end{aligned}$$

计算差分表的函数:

```
% 计算差分表的函数
function delta_f = differences(y)
    % 获取数据点的数量
    n = length(y);
    % 初始化差分表
    delta_f = zeros(n, n);
    % 第一列为原始数据
    delta_f(:, 1) = y(:);

    % 计算差分表
    for j = 2:n
        for i = 1:n-j+1
            % 递归计算差分
            delta_f(i, j) = delta_f(i+1, j-1) - delta_f(i, j-1);
        end
    end
end
```

实现等距节点 Newton 前向插值公式的函数:

```
function newton_interpolation(f, n, interval)
    x_exact = linspace(interval(1), interval(2), 1000);
    x_nodes = linspace(interval(1), interval(2), n);
    y_exact = f(x_exact); % 精确值
    y_nodes = f(x_nodes); % 等距采样

    % 计算差分表
    delta_f = differences(y_nodes);
    % 计算节点间距 h
    h = x_nodes(2) - x_nodes(1);
    % 计算 p
    p = (x_exact - x_nodes(1)) / h;

    % 初始化插值结果为差分表的第一项
    y_interp = delta_f(1) * ones(size(x_exact));
    % 初始化二项式系数
    binom = 1;

    % 遍历差分表, 累加插值项
    for k = 1:n-1
        % 更新二项式系数
        binom = binom .* (p-(k-1)) / k;
        % 累加插值项
        y_interp = y_interp + delta_f(1, k+1) * binom;
    end

    % 插值与精确值的差距
    y_diff = abs(y_interp - y_exact);

    % 绘图
    % 左侧纵轴: 绘制精确值和插值结果
    yyaxis left;
    plot(x_exact, y_exact, 'k-', 'Linewidth', 1.5); hold on;
    plot(x_exact, y_interp, 'r--', 'Linewidth', 1);
    scatter(x_nodes, y_nodes, 25, 'k', 'filled');
    title(['Newton Interpolation with n = ', num2str(n)]);
    hold off;

    % 右侧纵轴: 绘制 log(y_diff)
    yyaxis right;
    plot(x_exact, log(y_diff), 'b-.', 'Linewidth', 1);
    ylabel('log(y_{diff})');
    legend('Exact', 'Interpolated', 'Nodes', 'log difference');
    grid on;
end
```

函数调用:

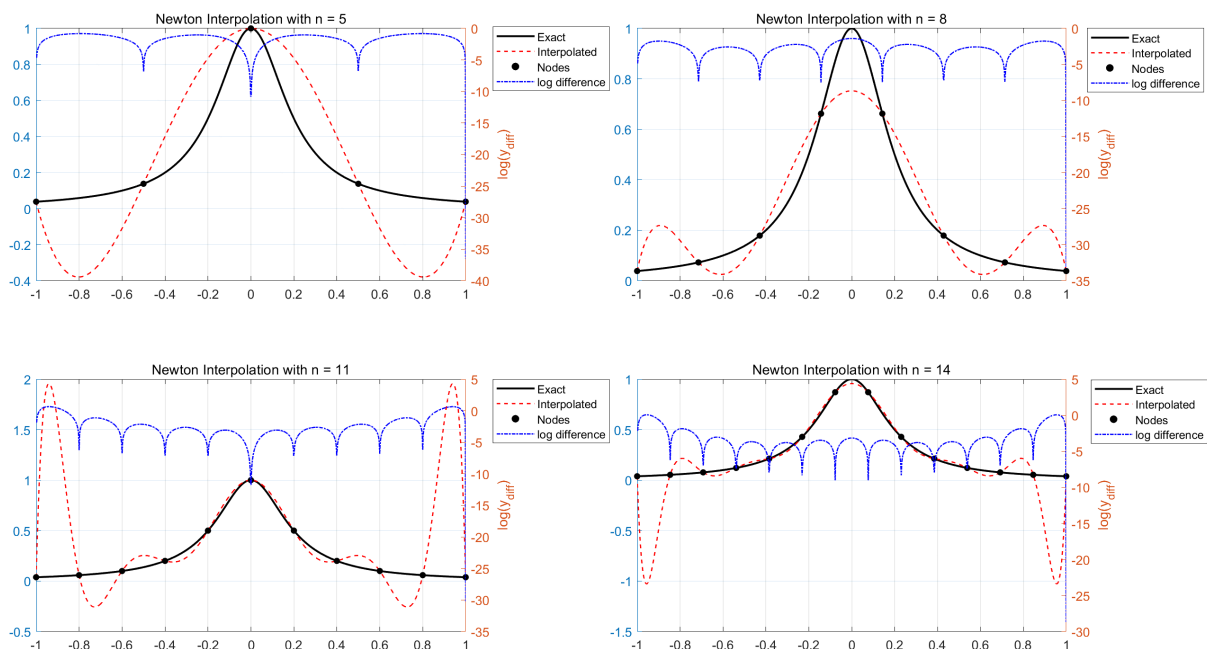
```

f = @(x) 1 ./ (1 + 25 * x.^2);
ns_f= [5, 8, 11, 14];

figure;
for i = 1:length(ns_f)
    subplot(2,2,i);
    newton_interpolation(f, ns_f(i), [-1, 1]);
end

```

运行结果:



(2) Chebyshev 节点的 Newton 插值

第一类 Chebyshev 多项式:

$$\begin{aligned}
 T_n(x) &= \cos[n \arccos(x)] \\
 &= \frac{(x + \sqrt{x^2 - 1})^n + (x - \sqrt{x^2 - 1})^n}{2} \quad (-1 \leq x \leq 1)
 \end{aligned}$$

这个 n 次多项式在 $[-1, 1]$ 区间上的 n 个根为 $\cos(\frac{2k-1}{2n}\pi)$ ($k = 1, \dots, n$)

计算差商表的函数:

```

% 计算差商表的函数
function delta_f = divided_differences(x, y)
    % 获取数据点的数量
    n = length(y);
    % 初始化差商表
    delta_f = zeros(n, n);
    % 第一列为原始数据
    delta_f(:, 1) = y(:);

    % 计算差商表
    for j = 2:n
        for i = 1:n-j+1
            % 递归计算差商

```

```

        delta_f(i, j) = (delta_f(i+1, j-1) - delta_f(i, j-1)) / (x(j+i-1)-x(i));
    end
end

% 返回差商表的第一行
delta_f = delta_f(1,:);
end

```

实现一般的 Newton 插值公式的函数:

```

function newton_interpolation(f, n, interval)
    x_exact = linspace(interval(1), interval(2), 1000);
    y_exact = f(x_exact); % 精确值

    index = 1:n;
    x_nodes = cos((2*index-1) * pi / (2*n));
    y_nodes = f(x_nodes); % Chebyshev 节点采样

    % 计算差商表 (的第一行)
    delta_f = divided_differences(x_nodes, y_nodes);

    % 初始化插值结果和累乘项
    y_interp = zeros(size(x_exact));
    term = ones(size(x_exact));

    % Newton 插值公式
    for j = 1:length(y_nodes)
        y_interp = y_interp + delta_f(j) * term; % 累加插值项
        term = term .* (x_exact - x_nodes(j)); % 更新累乘项
    end

    % 插值与精确值的差距
    y_diff = abs(y_interp - y_exact);

    % 绘图
    % 左侧纵轴: 绘制精确值和插值结果
    yyaxis left;
    plot(x_exact, y_exact, 'k-', 'Linewidth', 1.5); hold on;
    plot(x_exact, y_interp, 'r--', 'Linewidth', 1);
    scatter(x_nodes, y_nodes, 25, 'k', 'filled');
    title(['Newton Interpolation with n = ', num2str(n)]);
    hold off;

    % 右侧纵轴: 绘制 log(y_diff)
    yyaxis right;
    plot(x_exact, log(y_diff), 'b-.', 'Linewidth', 1);
    ylabel('log(y_{diff})');
    legend('Exact', 'Interpolated', 'Nodes', 'log difference');
    grid on;
end

```

函数调用:

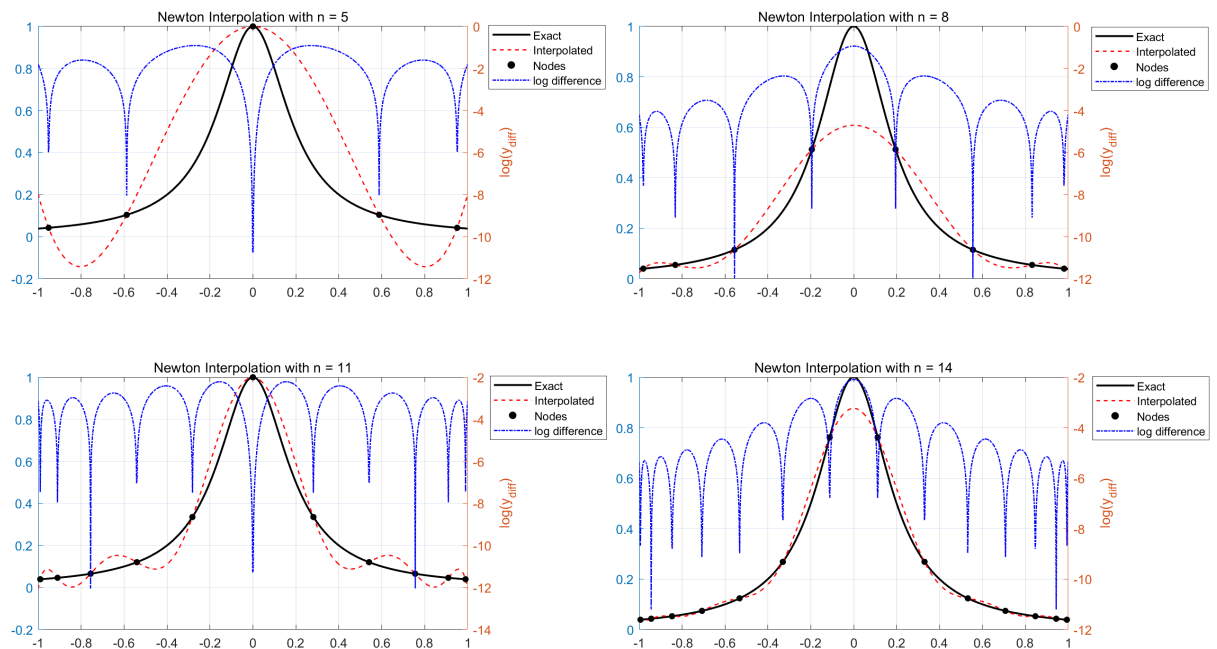
```

f = @(x) 1 ./ (1 + 25 * x.^2);
ns_f= [5, 8, 11, 14];

figure;
for i = 1:length(ns_f)
    subplot(2,2,i);
    newton_interpolation(f, ns_f(i), [-1, 1]);
end

```

运行结果:



(3) 等距节点的三次 Hermite 插值

给定 2 个点的 0, 1 阶导数信息:

$$\begin{aligned}
 &(x_1, f(x_1)), (x_1, f'(x_1)) \\
 &(x_2, f(x_2)), (x_2, f'(x_2))
 \end{aligned}$$

其差商表为:

x_1	x_1	x_2	x_2
$f(x_1)$	$f(x_1)$	$f(x_2)$	$f(x_2)$
	$f'(x_1)/1!$	$f[x_1, x_2]$	$f'(x_2)/1!$
		$f[x_1, x_1, x_2]$	$f[x_1, x_2, x_2]$
			$f[x_1, x_1, x_2, x_2]$

对应的 Hermite 插值公式为:

$$\begin{aligned}
P(x) &:= f(x_1) + f'(x_1)(x - x_1) + f[x_1, x_1, x_2](x - x_1)^2 + f[x_1, x_1, x_2, x_2](x - x_1)^2(x - x_2) \\
f[x_1, x_2] &= \frac{f(x_2) - f(x_1)}{x_2 - x_1} \\
f[x_1, x_1, x_2] &= \frac{f[x_1, x_2] - f'(x_1)}{x_2 - x_1} \\
f[x_1, x_2, x_2] &= \frac{f'(x_2) - f[x_1, x_2]}{x_2 - x_1} \\
f[x_1, x_1, x_2, x_2] &= \frac{f[x_1, x_2, x_2] - f[x_1, x_1, x_2]}{x_2 - x_1}
\end{aligned}$$

但我们很难一下子从中看出什么来，还是沿用邵老师的方法吧，构造以下函数：

$$\begin{aligned}
\phi(x) &:= 3x^2 - 2x^3 \\
\varphi(x) &:= x^3 - x^2
\end{aligned}$$

容易验证：

$$\begin{aligned}
\phi(0) &= \phi'(0) = 0 \\
\phi(1) &= 1, \phi'(1) = 0 \\
\varphi(0) &= \varphi'(0) = 0 \\
\varphi(1) &= 0, \varphi'(1) = 1
\end{aligned}$$

于是 Hermite 插值多项式具有如下形式：

$$\begin{aligned}
P(x) &:= f(x_1)\phi\left(\frac{x_2 - x}{x_2 - x_1}\right) + f(x_2)\phi\left(\frac{x - x_1}{x_2 - x_1}\right) \\
&\quad - f'(x_1)(x_2 - x_1)\varphi\left(\frac{x_2 - x}{x_2 - x_1}\right) + f'(x_2)(x_2 - x_1)\varphi\left(\frac{x - x_1}{x_2 - x_1}\right) \\
P(x_1) &= f(x_1)\phi(1) + f(x_2)\phi(0) - f'(x_1)(x_2 - x_1)\varphi(1) + f'(x_2)(x_2 - x_1)\varphi(0) \\
&= f(x_1) \\
P(x_2) &= f(x_1)\phi(0) + f(x_2)\phi(1) - f'(x_1)(x_2 - x_1)\varphi(0) + f'(x_2)(x_2 - x_1)\varphi(1) \\
&= f(x_2) \\
P'(x) &:= -\frac{f(x_1)}{x_2 - x_1}\phi'\left(\frac{x_2 - x}{x_2 - x_1}\right) + \frac{f(x_2)}{x_2 - x_1}\phi'\left(\frac{x - x_1}{x_2 - x_1}\right) \\
&\quad + f'(x_1)\varphi'\left(\frac{x_2 - x}{x_2 - x_1}\right) + f'(x_2)\varphi'\left(\frac{x - x_1}{x_2 - x_1}\right) \\
P'(x_1) &= -\frac{f(x_1)}{x_2 - x_1}\phi'(1) + \frac{f(x_2)}{x_2 - x_1}\phi'(0) + f'(x_1)\varphi'(1) + f'(x_2)\varphi'(0) \\
&= f'(x_1) \\
P'(x_2) &= -\frac{f(x_1)}{x_2 - x_1}\phi'(0) + \frac{f(x_2)}{x_2 - x_1}\phi'(1) + f'(x_1)\varphi'(0) + f'(x_2)\varphi'(1) \\
&= f'(x_2)
\end{aligned}$$

一般来说，对于拥有 0, 1 阶导数信息的节点 $x_1, x_2, \dots, x_n$

我们只需对每一对相邻节点应用 Hermite 三次插值公式即可（就不用通过构造重节点差分表来计算了）

实现三次 Hermite 插值公式的函数：

```
function Hermite_interpolation(f, f_derivative, phi, varphi, n, interval)
    x_exact = linspace(interval(1), interval(2), 100);
    x_nodes = linspace(interval(1), interval(2), n);
    y_exact = f(x_exact); % 精确值
    y_nodes = f(x_nodes); % 等距采样
    y_nodes_derivative = f_derivative(x_nodes);
```

```

% 节点间距
h = x_nodes(2) - x_nodes(1);

% 段序号
index = floor((x_exact - x_nodes(1)) / h) + 1;
index(end) = n - 1;

% Hermite 插值公式
y_interp = y_nodes(index) .* phi((x_nodes(index + 1) - x_exact) / h) + ...
           y_nodes(index + 1) .* phi((x_exact - x_nodes(index)) / h) - ...
           h * y_nodes_derivative(index) .* varphi((x_nodes(index + 1) - x_exact) / h) + ...
           h * y_nodes_derivative(index + 1) .* varphi((x_exact - x_nodes(index)) / h);

% 插值与精确值的差距
y_diff = abs(y_interp - y_exact);

% 绘图
% 左侧纵轴：绘制精确值和插值结果
yyaxis left;
plot(x_exact, y_exact, 'k-', 'Linewidth', 1.5); hold on;
plot(x_exact, y_interp, 'r--', 'Linewidth', 1);
scatter(x_nodes, y_nodes, 25, 'k', 'filled');
title(['Newton Interpolation with n = ', num2str(n)]);
hold off;

% 右侧纵轴：绘制 log(y_diff)
yyaxis right;
plot(x_exact, log(y_diff), 'b-.', 'Linewidth', 1);
ylabel('log(y_{diff})');
legend('Exact', 'Interpolated', 'Nodes', 'log difference');
grid on;
end

```

函数调用:

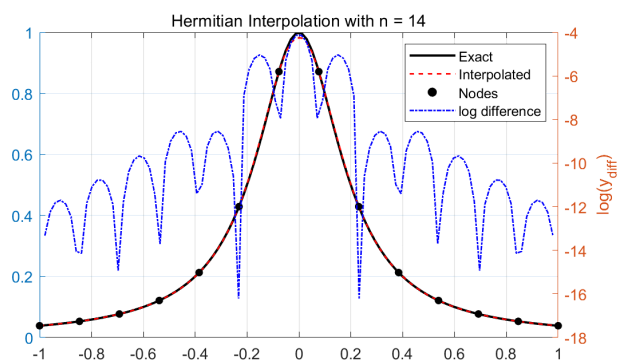
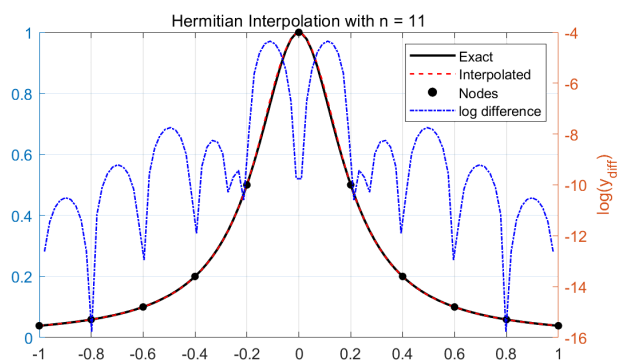
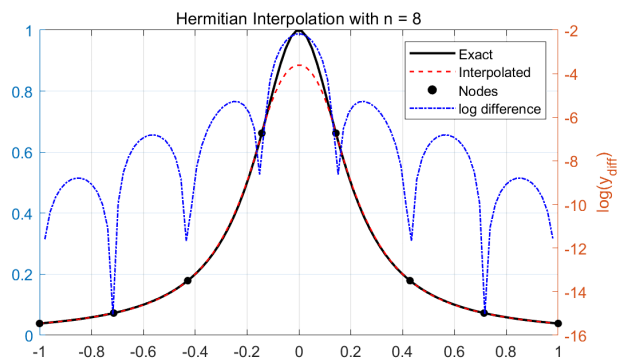
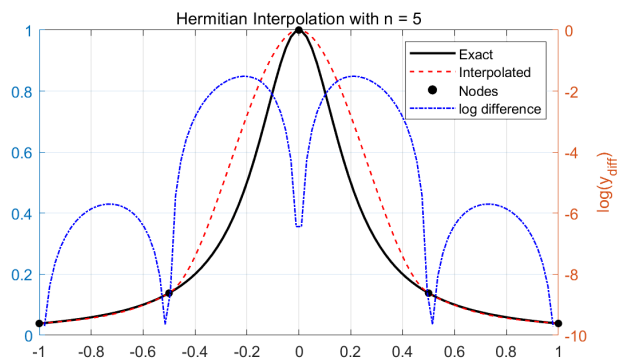
```

f = @(x) 1 ./ (1 + 25 * x.^2);
f_derivative = @(x) (-50 * x) ./ ((1 + 25 * x.^2).^2);
phi = @(x) 3 * x.^2 - 2 * x.^3;
varphi = @(x) x.^3 - x.^2;
ns_f= [5, 8, 11, 14];

figure;
for i = 1:length(ns_f)
    subplot(2,2,i);
    Hermite_interpolation(f, f_derivative, phi, varphi, ns_f(i), [-1, 1]);
end

```

运行结果:



Problem 2

Can a unique polynomial $p(x)$ of degree up to three always be found that satisfies the interpolation conditions $f(0)$, $f''(0)$, $f(1)$, and $f''(1)$?
If so, construct the interpolating polynomial.

- (DeepSeek 神力)

事实上，这个问题可以用待定系数法直接解，不用下面那种更有技巧性的解法。

To determine if a unique polynomial $p(x)$ of degree up to three can be found that satisfies the interpolation conditions $p(0)$, $p''(0)$, $p(1)$, and $p''(1)$, we start by considering the general form of a cubic polynomial $p(x) = ax^3 + bx^2 + cx + d$. The second derivative is $p''(x) = 6ax + 2b$.

Applying the given conditions, we get the following system of equations:

1. $p(0) = d$
2. $p''(0) = 2b$
3. $p(1) = a + b + c + d$
4. $p''(1) = 6a + 2b$

Solving these equations step by step:

1. From $p(0) = d$, we have $d = y_0$.
2. From $p''(0) = 2b$, we have $b = \frac{y_0''}{2}$.
3. From $p''(1) = 6a + 2b$, substituting b , we get $6a + y_0'' = y_1''$, leading to $a = \frac{y_1'' - y_0''}{6}$.
4. Substituting a , b , and d into $p(1) = a + b + c + d$, we solve for c :

$$c = y_1 - y_0 - \frac{y_1'' + 2y_0''}{6}$$

Thus, the coefficients a , b , c , and d are uniquely determined by the given conditions, confirming the existence of a unique cubic polynomial. The matrix formed by these conditions has a non-zero determinant, ensuring the system is invertible and hence has a unique solution.

The interpolating polynomial is constructed as follows:

$$p(x) = \frac{p''(1) - p''(0)}{6}x^3 + \frac{p''(0)}{2}x^2 + \left(p(1) - p(0) - \frac{p''(1) + 2p''(0)}{6}\right)x + p(0)$$

Solution:

受三次 Hermite 插值的启发, 我们知道只要有两个至多 3 次的多项式 $\phi(x)$ 和 $\varphi(x)$ 使得:

$$\begin{aligned} \phi(0) &= \phi''(0) = 0 \\ \phi(1) &= 1, \phi''(1) = 0 \\ \varphi(0) &= \varphi''(0) = 0 \\ \varphi(1) &= 0, \varphi''(1) = 1 \end{aligned}$$

则对于给定的 $f(0)$, $f''(0)$ 和 $f(1)$, $f''(1)$ (假设 $x_1 \neq x_2$),

我们就能构造如下至多 3 次的插值多项式:

$$\begin{aligned} P(x) &:= f(0)\phi(1-x) + f(1)\phi(x) + f''(0)\varphi(1-x) + f''(1)\varphi(x) \\ P''(x) &:= f(0)\phi''(1-x) + f(1)\phi''(x) + f''(0)\varphi''(1-x) + f''(1)\varphi''(x) \\ \hline P(0) &= f(0)\phi(1) + f(1)\phi(0) + f''(0)\varphi(1) + f''(1)\varphi(0) \\ &= f(0) \\ P(1) &= f(0)\phi(0) + f(1)\phi(1) + f''(0)\varphi(0) + f''(1)\varphi(1) \\ &= f(1) \\ P''(0) &= f(0)\phi''(1) + f(1)\phi''(0) + f''(0)\varphi''(1) + f''(1)\varphi''(0) \\ &= f''(0) \\ P''(1) &= f(0)\phi''(0) + f(1)\phi''(1) + f''(0)\varphi''(0) + f''(1)\varphi''(1) \\ &= f''(1) \end{aligned}$$

于是问题归结为是否存在一对至多 3 次的多项式 $\phi(x)$ 和 $\varphi(x)$ 使得:

$$\begin{array}{c}
\phi(0) = \phi''(0) = 0 \\
\phi(1) = 1, \phi''(1) = 0 \\
\hline
\varphi(0) = \varphi''(0) = 0 \\
\varphi(1) = 0, \varphi''(1) = 1 \\
\hline
\phi(x), \phi(1-x), \varphi(x), \varphi(1-x) \text{ are linearly independent}
\end{array}$$

- ① 根据 $\phi''(0) = \phi''(1) = 0$ 易知 $\phi(x)$ 是至多一次的多项式.
再根据 $\phi(0) = 0$ 和 $\phi(1) = 1$ 可知 $\phi(x) = x$
- ② 根据 $\varphi''(0) = 0$ 和 $\varphi''(1) = 1$ 可知 $\varphi(x)$ 是三次多项式, 且 $\varphi''(x) = x$
再根据 $\varphi(0) = 0$ 和 $\varphi(1) = 0$ 可知 $\varphi(x) = \frac{1}{6}(x^3 - x)$
- ③ 显然 $\begin{cases} \phi(x) = x \\ \phi(1-x) = 1-x \\ \varphi(x) = \frac{1}{6}(x^3 - x) \\ \varphi(1-x) = -\frac{1}{6}(x^3 - 3x^2 + 2x) \end{cases}$ 线性无关, 构成线性空间 $\text{span}\{1, x, x^2, x^3\}$ 的一组基

因此存在唯一的至多三次的插值多项式 $P(x)$ 满足 $\begin{cases} P(0) = f(0), P(1) = f(1) \\ P''(0) = f''(0), P''(1) = f''(1) \end{cases}$

$$\begin{aligned}
P(x) &:= f(0)\phi(1-x) + f(1)\phi(x) + f''(0)\varphi(1-x) + f''(1)\varphi(x) \\
&= f(0)(1-x) + f(1)x + f''(0) \cdot \frac{1}{6}(-x^3 + 3x^2 - 2x) + f''(1) \cdot \frac{1}{6}(x^3 - x) \\
&= \frac{f''(1) - f''(0)}{6}x^3 + \frac{f''(0)}{2}x^2 + \left(f(1) - f(0) - \frac{f''(1) + 2f''(0)}{6}\right)x + f(0)
\end{aligned}$$

Problem 3

Similar to complete, natural, and periodic cubic splines,
when the "not-a-knot" condition is used in cubic spline interpolation,
the computational kernel is also to solve a sparse linear system.
Try to derive the corresponding linear system.

Solution:

设样条 $s(x)$ 在区间 $[a, b]$ 上, 型值点 (x_i, y_i) ($i = 0, \dots, n$) 的横坐标为 $a = x_0 < x_1 < \dots < x_n = b$
假设 $s(x)$ 在相邻型值点之间为三次多项式, 即 $s(x)$ 在区间 $[x_i, x_{i+1}]$ 上的表达式为:

$$s_i(x) := a_i + b_i x + c_i x^2 + d_i x^3 \quad (i = 0, \dots, n-1)$$

因此 $s(x)$ 在整个 $[a, b]$ 区间中有 $4n$ 个待定系数.

它要满足以下条件:

- ① 通过型值点: (共 $2n$ 个方程)

$$\begin{cases} s_i(x_i) = y_i \\ s_i(x_{i+1}) = y_{i+1} \end{cases} \quad (i = 0, \dots, n-1)$$

- ② 在型值点处的一阶导数连续: (共 $n-1$ 个方程)

$$s'_i(x_{i+1}) = s'_{i+1}(x_{i+1}) \quad (i = 0, \dots, n-2)$$

- ③ 在型值点处的二阶导数连续: (共 $n-1$ 个方程)

$$s''_i(x_{i+1}) = s''_{i+1}(x_{i+1}) \quad (i = 0, \dots, n-2)$$

- ④ Not-a-knot 边界条件: (共 2 个方程)

消除样条在 x_1 和 x_{n-1} 处的 "结", 即要求三阶导数在 x_1 和 x_{n-1} 处连续.

换言之, 我们要求 $[x_0, x_1]$ 上的三次多项式 $s_0(x)$ 和 $[x_1, x_2]$ 上的三次多项式 $s_1(x)$ 相同,
同时要求 $[x_{n-2}, x_{n-1}]$ 上的三次多项式 $s_{n-2}(x)$ 和 $[x_{n-1}, x_n]$ 上的三次多项式 $s_{n-1}(x)$ 相同.

$$\begin{aligned}s_0'''(x_1) &= s_1'''(x_1) \\ s_{n-2}'''(x_{n-1}) &= s_{n-1}'''(x_{n-1})\end{aligned}$$

求解 $4n$ 阶线性方程组的代价 $O(n^3)$ 还是太大了.
我们需要想办法将 $4n$ 阶线性方程组变为低阶线性方程组.

注意到我们有 $n+1$ 个导数值 $s'(x_0), \dots, s'(x_n)$ 未知.
我们可以利用它们进行三次 Hermite 插值.

定义:

$$\begin{aligned}k_i &:= s'(x_i) & (i = 0, 1, \dots, n) \\ h_i &:= x_{i+1} - x_i & (i = 0, 1, \dots, n) \\ \phi(x) &:= 3x^2 - 2x^3 \\ \varphi(x) &:= x^3 - x^2\end{aligned}$$

其中 $k_0, k_1, \dots, k_{n-1}, k_n$ 都是未知的.

于是根据插值条件 $\begin{cases} s_i(x_i) = y_i \\ s_i(x_{i+1}) = y_{i+1} \\ s_i'(x_i) = k_i \\ s_i'(x_{i+1}) = k_{i+1} \end{cases}$ 和三次 Hermite 插值的结论可得 $s_i(x)$ 的表达式为:

$$\begin{aligned}s_i(x) &= y_i \phi\left(\frac{x_{i+1}-x}{h_i}\right) + y_{i+1} \phi\left(\frac{x-x_i}{h_i}\right) - k_i h_i \varphi\left(\frac{x_{i+1}-x}{h_i}\right) + k_{i+1} h_i \varphi\left(\frac{x-x_i}{h_i}\right) \\ s_i'(x) &= -\frac{y_i}{h_i} \phi'\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{y_{i+1}}{h_i} \phi'\left(\frac{x-x_i}{h_i}\right) + k_i \varphi'\left(\frac{x_{i+1}-x}{h_i}\right) + k_{i+1} \varphi'\left(\frac{x-x_i}{h_i}\right) \\ s_i''(x) &= \frac{y_i}{h_i^2} \phi''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{y_{i+1}}{h_i^2} \phi''\left(\frac{x-x_i}{h_i}\right) - \frac{k_i}{h_i} \varphi''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{k_{i+1}}{h_i} \varphi''\left(\frac{x-x_i}{h_i}\right) \\ s_i'''(x) &= -\frac{y_i}{h_i^3} \phi'''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{y_{i+1}}{h_i^3} \phi'''\left(\frac{x-x_i}{h_i}\right) + \frac{k_i}{h_i^2} \varphi'''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{k_{i+1}}{h_i^2} \varphi'''\left(\frac{x-x_i}{h_i}\right) \\ &\equiv 12 \frac{y_i - y_{i+1}}{h_i^3} + 6 \frac{k_i + k_{i+1}}{h_i^2} & (i = 0, 1, \dots, n-1) \\ &= -4\beta_i \eta_i + 6\beta_i^2(k_i + k_{i+1})\end{aligned}$$

where $\phi(x) = 3x^2 - 2x^3$, $\varphi(x) = x^3 - x^2$

$$\phi'(x) = 6x - 6x^2, \quad \varphi'(x) = 3x^2 - 2x$$

$$\phi''(x) = 6 - 12x, \quad \varphi''(x) = 6x - 2$$

$$\phi'''(x) \equiv -12, \quad \varphi'''(x) \equiv 6$$

$$\beta_i = \frac{1}{h_i}, \quad \eta_i = 3 \frac{y_{i+1} - y_i}{h_i^2}$$

现考虑二阶导的插值条件 $s_i''(x_{i+1}) = s_{i+1}''(x_{i+1})$:

$$\begin{aligned}
s''_i(x_{i+1}) &= \frac{y_i}{h_i^2} \phi''(0) + \frac{y_{i+1}}{h_i^2} \phi''(1) - \frac{k_i}{h_i} \varphi''(0) + \frac{k_{i+1}}{h_i} \varphi''(1) \\
&= 6 \frac{y_i}{h_i^2} - 6 \frac{y_{i+1}}{h_i^2} + 2 \frac{k_i}{h_i} + 4 \frac{k_{i+1}}{h_i} \\
s''_{i+1}(x_{i+1}) &= \frac{y_{i+1}}{h_{i+1}^2} \phi''(1) + \frac{y_{i+2}}{h_{i+1}^2} \phi''(0) - \frac{k_{i+1}}{h_{i+1}} \varphi''(1) + \frac{k_{i+2}}{h_{i+1}} \varphi''(0) \\
&= -6 \frac{y_{i+1}}{h_{i+1}^2} + 6 \frac{y_{i+2}}{h_{i+1}^2} - 4 \frac{k_{i+1}}{h_{i+1}} - 2 \frac{k_{i+2}}{h_{i+1}} \\
\hline
s''_i(x_{i+1}) &= s''_{i+1}(x_{i+1}) \\
&\Downarrow \\
6 \frac{y_i}{h_i^2} - 6 \frac{y_{i+1}}{h_i^2} + 2 \frac{k_i}{h_i} + 4 \frac{k_{i+1}}{h_i} &= -6 \frac{y_{i+1}}{h_{i+1}^2} + 6 \frac{y_{i+2}}{h_{i+1}^2} - 4 \frac{k_{i+1}}{h_{i+1}} - 2 \frac{k_{i+2}}{h_{i+1}} \\
&\Downarrow \\
\beta_i k_i + \alpha_{i+1} k_{i+1} + \beta_{i+1} k_{i+2} &= \eta_i + \eta_{i+1} \\
\text{where } \begin{cases} h_i = x_{i+1} - x_i & (i = 0, \dots, n-1) \\ \beta_i = \frac{1}{h_i} & (i = 0, \dots, n-1) \\ \alpha_{i+1} = 2(\beta_i + \beta_{i+1}) & (i = 0, \dots, n-2) \\ \eta_i = 3 \frac{y_{i+1} - y_i}{h_i^2} & (i = 0, \dots, n-1) \end{cases}
\end{aligned}$$

现考虑 Not-a-knot 样条边界条件:

消除样条在 x_1 和 x_{n-1} 处的 "结", 即要求三阶导数在 x_1 和 x_{n-1} 处连续.

换言之, 我们要求 $[x_0, x_1]$ 上的三次多项式 $s_0(x)$ 和 $[x_1, x_2]$ 上的三次多项式 $s_1(x)$ 相同,

同时要求 $[x_{n-2}, x_{n-1}]$ 上的三次多项式 $s_{n-2}(x)$ 和 $[x_{n-1}, x_n]$ 上的三次多项式 $s_{n-1}(x)$ 相同.

$$\begin{aligned}
s'''_0(x_1) &= s'''_1(x_1) \\
s'''_{n-2}(x_{n-1}) &= s'''_{n-1}(x_{n-1}) \\
&\Downarrow \\
-4\beta_0\eta_0 + 6\beta_0^2(k_0 + k_1) &= s'''_0(x_1) = s'''_1(x_1) = -4\beta_1\eta_1 + 6\beta_1^2(k_1 + k_2) \\
-4\beta_{n-2}\eta_{n-2} + 6\beta_{n-2}^2(k_{n-2} + k_{n-1}) &= s'''_{n-2}(x_{n-1}) = s'''_{n-1}(x_{n-1}) = -4\beta_{n-1}\eta_{n-1} + 6\beta_{n-1}^2(k_{n-1} + k_n) \\
&\Downarrow \\
3\beta_0^2 k_0 + 3(\beta_0^2 - \beta_1^2)k_1 - 3\beta_1^2 k_2 &= 2(\beta_0\eta_0 - \beta_1\eta_1) \\
3\beta_{n-2}^2 k_{n-2} + 3(\beta_{n-2}^2 - \beta_{n-1}^2)k_{n-1} - 3\beta_{n-1}^2 k_n &= 2(\beta_{n-2}\eta_{n-2} - \beta_{n-1}\eta_{n-1}) \\
&\Downarrow \\
3\beta_0 k_0 + 3(\beta_0 - \frac{\beta_1^2}{\beta_0})k_1 - 3\frac{\beta_1^2}{\beta_0}k_2 &= 2(\eta_0 - \frac{\beta_1}{\beta_0}\eta_1) \\
3\beta_{n-2} k_{n-2} + 3(\beta_{n-2} - \frac{\beta_{n-1}^2}{\beta_{n-2}})k_{n-1} - 3\frac{\beta_{n-1}^2}{\beta_{n-2}}k_n &= 2(\eta_{n-2} - \frac{\beta_{n-1}}{\beta_{n-2}}\eta_{n-1})
\end{aligned}$$

于是方程组为:

$$\begin{cases} 3\beta_0 k_0 + 3(\beta_0 - \frac{\beta_1^2}{\beta_0})k_1 - 3\frac{\beta_1^2}{\beta_0}k_2 = 2(\eta_0 - \frac{\beta_1}{\beta_0}\eta_1) & (\text{additional condition}) \\ \beta_i k_i + \alpha_{i+1}k_{i+1} + \beta_{i+1}k_{i+2} = \eta_i + \eta_{i+1} & (i = 0, \dots, n-2) \\ 3\beta_{n-2}k_{n-2} + 3(\beta_{n-2} - \frac{\beta_{n-1}^2}{\beta_{n-2}})k_{n-1} - 3\frac{\beta_{n-1}^2}{\beta_{n-2}}k_n = 2(\eta_{n-2} - \frac{\beta_{n-1}}{\beta_{n-2}}\eta_{n-1}) & (\text{additional condition}) \end{cases}$$

$\Updownarrow$

$$\begin{bmatrix} 3\beta_0 & 3(\beta_0 - \frac{\beta_1^2}{\beta_0}) & -3\frac{\beta_1^2}{\beta_0} & & & \\ \beta_0 & \alpha_1 & \beta_1 & & & \\ & \beta_1 & \alpha_2 & \beta_2 & & \\ & & \beta_2 & \ddots & \ddots & \\ & & & \ddots & \alpha_{n-2} & \beta_{n-2} \\ & & & & \beta_{n-2} & \alpha_{n-1} & \beta_{n-1} \\ & & & & & 3\beta_{n-2} & 3(\beta_{n-2} - \frac{\beta_{n-1}^2}{\beta_{n-2}}) & -3\frac{\beta_{n-1}^2}{\beta_{n-2}} \end{bmatrix} \begin{bmatrix} k_0 \\ k_1 \\ k_2 \\ \vdots \\ k_{n-2} \\ k_{n-1} \\ k_n \end{bmatrix} = \begin{bmatrix} 2(\eta_0 - \frac{\beta_1}{\beta_0}\eta_1) \\ \eta_0 + \eta_1 \\ \eta_1 + \eta_2 \\ \vdots \\ \eta_{n-3} + \eta_{n-2} \\ \eta_{n-2} + \eta_{n-1} \\ 2(\eta_{n-2} - \frac{\beta_{n-1}}{\beta_{n-2}}\eta_{n-1}) \end{bmatrix}$$

尽管这并不是一个严格对角占优的 $n+1$ 阶稀疏矩阵，但使用 Gauss 消去法求解的代价仍为 $O(n)$

假设其能进行不选主元的 Gauss 消元，则其 LU 分解为：

$$\begin{bmatrix} \times & \times & \boxed{\times} & & & \\ \times & \times & \times & & & \\ & \times & \times & \times & & \\ & & \times & \times & \times & \\ & & & \boxed{\times} & \times & \times \end{bmatrix} = \begin{bmatrix} \times & & & & & \\ \times & \times & & & & \\ & \times & \times & & & \\ & & \times & \times & & \\ & & & \boxed{\times} & \times & \times \end{bmatrix} \begin{bmatrix} \times & \times & \boxed{\times} & & & \\ & \times & \times & & & \\ & & \times & \times & & \\ & & & \times & \times & \\ & & & & \times & \times \\ & & & & & \times \end{bmatrix}$$

大部分列只需一次行消元， $O(1)$ 级别代价
Gauss 消元法总代价 $O(n)$
回代法和前代法各自代价为 $O(n)$
因此求解该稀疏线性系统的总代价为 $O(n)$

Problem 4

When performing interpolation with a complete cubic spline, the choice of derivatives on the boundary is important.

Suppose that Joe wants to interpolate the sine function $f(x) = \sin(x)$

at nine equispaced nodes over $[0, 2\pi]$, with $f'(0) = f'(2\pi) = 1$.

Unfortunately, he made a typo on $f'(0)$ in his program and observed some strange results.

Try to reproduce Joe's result with a few different values of $f'(0)$.

For instance, $f'(0) = 0$, $f'(0) = -1$, etc.

Solution:

设样条 $s(x)$ 在区间 $[a, b]$ 上，型值点 (x_i, y_i) ($i = 0, \dots, n$) 的横坐标为 $a = x_0 < x_1 < \dots < x_n = b$

假设 $s(x)$ 在相邻型值点之间为三次多项式，即 $s(x)$ 在区间 $[x_i, x_{i+1}]$ 上的表达式为：

$$s_i(x) := a_i + b_i x + c_i x^2 + d_i x^3 \quad (i = 0, \dots, n-1)$$

因此 $s(x)$ 在整个 $[a, b]$ 区间中有 $4n$ 个待定系数。

它要满足以下条件：

- ① 通过型值点：(共 $2n$ 个方程)

$$\begin{cases} s_i(x_i) = y_i \\ s_i(x_{i+1}) = y_{i+1} \end{cases} \quad (i = 0, \dots, n-1)$$

- ② 在型值点处的一阶导数连续: (共 $n-1$ 个方程)

$$s'_i(x_{i+1}) = s'_{i+1}(x_{i+1}) \quad (i = 0, \dots, n-2)$$

- ③ 在型值点处的二阶导数连续: (共 $n-1$ 个方程)

$$s''_i(x_{i+1}) = s''_{i+1}(x_{i+1}) \quad (i = 0, \dots, n-2)$$

- ④ 完全样条 (Complete Spline) 边界条件: (共 2 个方程)

假设已知样条在边界点 x_0, x_n 处的切线斜率值分别为 k_0, k_n .

$$\begin{aligned} s'_0(x_0) &= k_0 \\ s'_{n-1}(x_n) &= k_n \end{aligned}$$

求解 $4n$ 阶线性方程组的代价 $O(n^3)$ 还是太大了.

我们需要想办法将 $4n$ 阶线性方程组变为低阶线性方程组.

注意到我们有 $n+1$ 个导数值 $s'(x_0), \dots, s'(x_n)$ 未知.

我们可以利用它们进行三次 Hermite 插值.

定义:

$$\begin{aligned} k_i &:= s'(x_i) & (i = 0, 1, \dots, n) \\ h_i &:= x_{i+1} - x_i & (i = 0, 1, \dots, n) \\ \phi(x) &:= 3x^2 - 2x^3 \\ \varphi(x) &:= x^3 - x^2 \end{aligned}$$

其中 $k_0, k_1, \dots, k_{n-1}, k_n$ 都是未知的.

于是根据插值条件 $\begin{cases} s_i(x_i) = y_i \\ s_i(x_{i+1}) = y_{i+1} \\ s'_i(x_i) = k_i \\ s'_i(x_{i+1}) = k_{i+1} \end{cases}$ 和三次 Hermite 插值的结论可得 $s_i(x)$ 的表达式为:

$$\begin{aligned} s_i(x) &= y_i \phi\left(\frac{x_{i+1}-x}{h_i}\right) + y_{i+1} \phi\left(\frac{x-x_i}{h_i}\right) - k_i h_i \varphi\left(\frac{x_{i+1}-x}{h_i}\right) + k_{i+1} h_i \varphi\left(\frac{x-x_i}{h_i}\right) \\ s'_i(x) &= -\frac{y_i}{h_i} \phi'\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{y_{i+1}}{h_i} \phi'\left(\frac{x-x_i}{h_i}\right) + k_i \varphi'\left(\frac{x_{i+1}-x}{h_i}\right) + k_{i+1} \varphi'\left(\frac{x-x_i}{h_i}\right) \\ s''_i(x) &= \frac{y_i}{h_i^2} \phi''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{y_{i+1}}{h_i^2} \phi''\left(\frac{x-x_i}{h_i}\right) - \frac{k_i}{h_i} \varphi''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{k_{i+1}}{h_i} \varphi''\left(\frac{x-x_i}{h_i}\right) \\ s'''_i(x) &= -\frac{y_i}{h_i^3} \phi'''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{y_{i+1}}{h_i^3} \phi'''\left(\frac{x-x_i}{h_i}\right) + \frac{k_i}{h_i^2} \varphi'''\left(\frac{x_{i+1}-x}{h_i}\right) + \frac{k_{i+1}}{h_i^2} \varphi'''\left(\frac{x-x_i}{h_i}\right) \\ &\equiv 12 \frac{y_i - y_{i+1}}{h_i^3} + 6 \frac{k_i + k_{i+1}}{h_i^2} & (i = 0, 1, \dots, n-1) \\ &= -4\beta_i \eta_i + 6\beta_i^2 (k_i + k_{i+1}) \end{aligned}$$

where $\phi(x) = 3x^2 - 2x^3$, $\varphi(x) = x^3 - x^2$

$$\phi'(x) = 6x - 6x^2, \quad \varphi'(x) = 3x^2 - 2x$$

$$\phi''(x) = 6 - 12x, \quad \varphi''(x) = 6x - 2$$

$$\phi'''(x) \equiv -12, \quad \varphi'''(x) \equiv 6$$

$$\beta_i = \frac{1}{h_i}, \quad \eta_i = 3 \frac{y_{i+1} - y_i}{h_i^2}$$

现考虑二阶导的插值条件 $s''_i(x_{i+1}) = s''_{i+1}(x_{i+1})$:

$$\begin{aligned}
s_i''(x_{i+1}) &= \frac{y_i}{h_i^2} \phi''(0) + \frac{y_{i+1}}{h_i^2} \phi''(1) - \frac{k_i}{h_i} \varphi''(0) + \frac{k_{i+1}}{h_i} \varphi''(1) \\
&= 6 \frac{y_i}{h_i^2} - 6 \frac{y_{i+1}}{h_i^2} + 2 \frac{k_i}{h_i} + 4 \frac{k_{i+1}}{h_i} \\
s_{i+1}''(x_{i+1}) &= \frac{y_{i+1}}{h_{i+1}^2} \phi''(1) + \frac{y_{i+2}}{h_{i+1}^2} \phi''(0) - \frac{k_{i+1}}{h_{i+1}} \varphi''(1) + \frac{k_{i+2}}{h_{i+1}} \varphi''(0) \\
&= -6 \frac{y_{i+1}}{h_{i+1}^2} + 6 \frac{y_{i+2}}{h_{i+1}^2} - 4 \frac{k_{i+1}}{h_{i+1}} - 2 \frac{k_{i+2}}{h_{i+1}}
\end{aligned}$$

$$\begin{aligned}
s_i''(x_{i+1}) &= s_{i+1}''(x_{i+1}) \\
&\Downarrow \\
6 \frac{y_i}{h_i^2} - 6 \frac{y_{i+1}}{h_i^2} + 2 \frac{k_i}{h_i} + 4 \frac{k_{i+1}}{h_i} &= -6 \frac{y_{i+1}}{h_{i+1}^2} + 6 \frac{y_{i+2}}{h_{i+1}^2} - 4 \frac{k_{i+1}}{h_{i+1}} - 2 \frac{k_{i+2}}{h_{i+1}} \\
&\Downarrow \\
\beta_i k_i + \alpha_{i+1} k_{i+1} + \beta_{i+1} k_{i+2} &= \eta_i + \eta_{i+1}
\end{aligned}$$

where
$$\begin{cases}
h_i = x_{i+1} - x_i & (i = 0, \dots, n-1) \\
\beta_i = \frac{1}{h_i} & (i = 0, \dots, n-1) \\
\alpha_{i+1} = 2(\beta_i + \beta_{i+1}) & (i = 0, \dots, n-2) \\
\eta_i = 3 \frac{y_{i+1} - y_i}{h_i^2} & (i = 0, \dots, n-1)
\end{cases}$$

现考虑完全样条边界条件:

假设已知样条在边界点 x_0, x_n 处的切线斜率值分别为 k_0, k_n .

$$\begin{aligned}
s_0'(x_0) &= k_0 \\
s_{n-1}'(x_n) &= k_n
\end{aligned}$$

于是方程组为:

$$\begin{cases}
\alpha_1 k_1 + \beta_1 k_2 = \eta_0 + \eta_1 - \beta_0 k_0 & (\text{case of } i = 0) \\
\beta_i k_i + \alpha_{i+1} k_{i+1} + \beta_{i+1} k_{i+2} = \eta_i + \eta_{i+1} & (i = 1, \dots, n-3) \\
\beta_{n-2} k_{n-2} + \alpha_{n-1} k_{n-1} = \eta_{n-2} + \eta_{n-1} - \beta_n k_n & (\text{case of } i = n-2)
\end{cases}$$

$$\Downarrow$$

$$\begin{bmatrix}
\alpha_1 & \beta_1 & & & \\
\beta_1 & \alpha_2 & \beta_2 & & \\
& \beta_2 & \ddots & \ddots & \\
& & \ddots & \alpha_{n-2} & \beta_{n-2} \\
& & & \beta_{n-2} & \alpha_{n-1}
\end{bmatrix}
\begin{bmatrix}
k_1 \\
k_2 \\
\vdots \\
k_{n-2} \\
k_{n-1}
\end{bmatrix}
=
\begin{bmatrix}
\eta_0 + \eta_1 - \beta_0 k_0 \\
\eta_1 + \eta_2 \\
\vdots \\
\eta_{n-3} + \eta_{n-2} \\
\eta_{n-2} + \eta_{n-1} - \beta_n k_n
\end{bmatrix}$$

这是一个严格对角占优的 $n-1$ 阶三对角阵, 使用 Gauss 消去法求解的代价为 $O(n)$

严格对角占优意味着 Gauss 消去法不用选主元
于是三对角阵的 LU 分解为：

$$\begin{bmatrix} \times & \times & & & \\ \times & \times & \times & & \\ & \times & \times & \times & \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix} = \begin{bmatrix} \times & & & & \\ \times & \times & & & \\ & \times & \times & & \\ & & \times & \times & \\ & & & \times & \times \end{bmatrix} \begin{bmatrix} \times & \times & & & \\ & \times & \times & & \\ & & \times & \times & \\ & & & \times & \times \\ & & & & \times \end{bmatrix}$$

每列只需一次行消元, $O(1)$ 级别代价
Gauss 消元法总代价 $O(n)$
回代法和前代法各自代价为 $O(n)$
因此求解严格对角占优的 $n-1$ 阶三对角阵的代价为 $O(n)$

在等距节点的假设下，部分变量的形式可以得到简化：

$$x_{i+1} - x_i \equiv h$$

$$\beta = \frac{1}{h}$$

$$\alpha = 2(\beta + \beta) = 4\beta$$

此外，我们不引入严格对角占优的三对角阵的高效解法的细节，直接使用 MATLAB 内置操作符 `\` 进行求解。

实现等距节点的三次完全样条插值的函数：

```
function Complete_Cubic_Spline(f, phi, varphi, n, k1, kn, interval)
    x_exact = linspace(interval(1), interval(2), 1000);
    x_nodes = linspace(interval(1), interval(2), n);
    y_exact = f(x_exact); % 精确值
    y_nodes = f(x_nodes); % 等距采样

    % 构造三对角系统
    n_unknowns = n - 2;
    h = x_nodes(2) - x_nodes(1);
    beta = 1/h;
    A = diag(4*beta * ones(n_unknowns,1)) + ...
        diag(beta * ones(n_unknowns-1,1), -1) + ...
        diag(beta * ones(n_unknowns-1,1), 1);

    % 构造右端向量
    eta = 3 * (y_nodes(2:end) - y_nodes(1:end-1)) / h^2;
    b = eta(1:end-1) + eta(2:end);
    b(1) = b(1) - beta*k1;
    b(end) = b(end) - beta*kn; % 注意它是行向量

    % 求解内部导数
    k_internal = A \ b'; % 注意 k_internal 是列向量
    k = [k1; k_internal; kn]'; % 注意要保证 k 是行向量

    % 段序号
    index = floor((x_exact - x_nodes(1)) / h) + 1;
    index(end) = n - 1;

    % 分段计算样条值
    y_interp = y_nodes(index) .* phi((x_nodes(index + 1) - x_exact) / h) + ...
```



```

        y_nodes(index + 1) .* phi((x_exact - x_nodes(index)) / h) - ...
        h * k(index) .* varphi((x_nodes(index + 1) - x_exact) / h) + ...
        h * k(index + 1) .* varphi((x_exact - x_nodes(index)) / h);

% 插值与精确值的差距
y_diff = abs(y_interp - y_exact);

% 绘图
% 左侧纵轴：绘制精确值和插值结果
yyaxis left;
plot(x_exact, y_exact, 'k-', 'Linewidth', 1.5); hold on;
plot(x_exact, y_interp, 'r--', 'Linewidth', 1);
scatter(x_nodes, y_nodes, 25, 'k', 'filled');
title(['Complete Cubic Spline Interpolation with k1 = ', num2str(k1)]);
hold off;

% 右侧纵轴：绘制 log(y_diff)
yyaxis right;
plot(x_exact, log(y_diff), 'b-.', 'Linewidth', 1);
ylabel('log(y_{diff})');
legend('Exact', 'Interpolated', 'Nodes', 'log difference');
grid on;
end

```

函数调用:

```

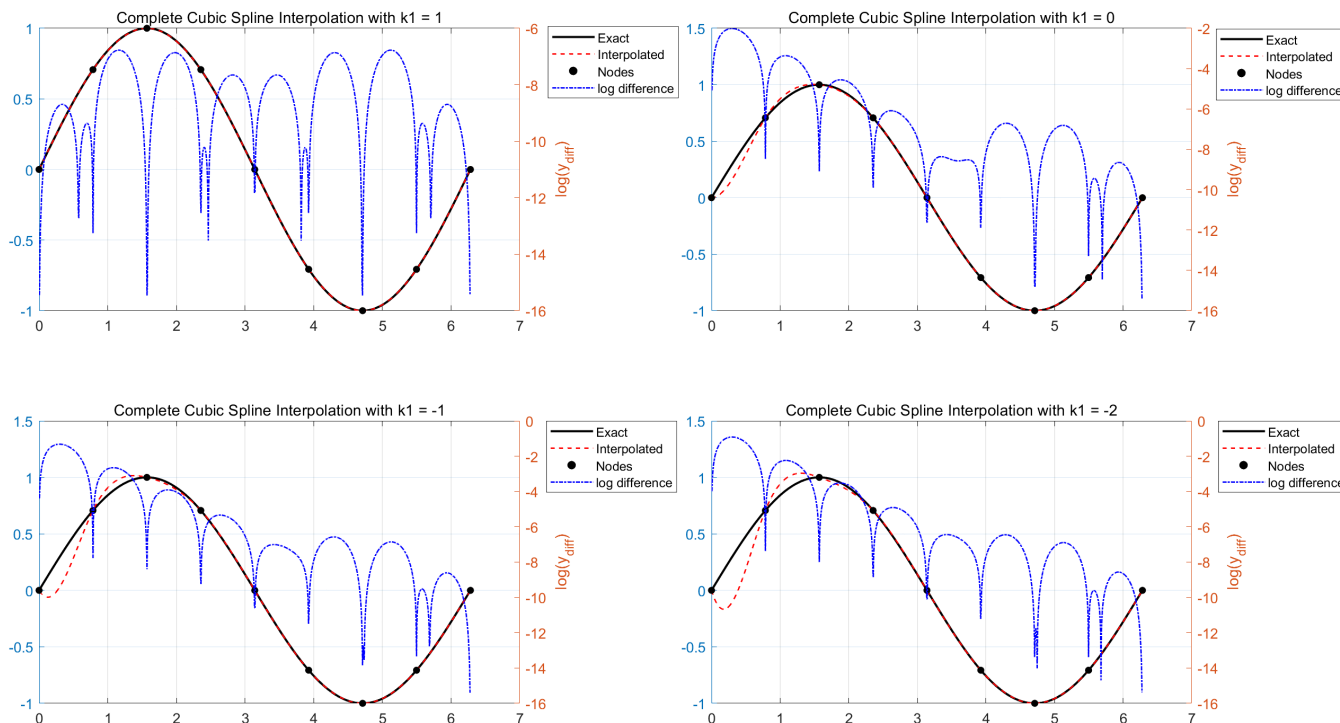
f = @(x) sin(x);
f_derivative = @(x) cos(x);
phi = @(x) 3 * x.^2 - 2 * x.^3;
varphi = @(x) x.^3 - x.^2;

% 设置边界导数
k1_values = [1, 0, -1, -2];
kn = f_derivative(2*pi); % 右边界导数使用精确值
n = 9;

figure;
for i = 1:length(k1_values)
    subplot(2,2,i);
    Complete_Cubic_Spline(f, phi, varphi, n, k1_values(i), kn, [0, 2*pi]);
end

```

运行结果:



三次样条插值的边界条件错误带来的影响一定是全局的，但到远端会逐步衰减。

这是因为此问题的系数矩阵 A 的逆矩阵 A^{-1} 的元素越靠近主对角线就越大。

误差是逆矩阵 A^{-1} 乘右端项 b ，于是右端项第一个元素的偏差对解的第一个元素影响大，对解的后续元素影响小。

但 B 样条只会在局部有影响。

Problem 5

The temperature in the human body is not a constant,

but rather follows a daily rhythm driven by an internal biological clock.

The following table lists 20 averaged values of temperature measurements taken from 70 English sailors in an experiment done in 1971.

Time (hour)	Temperature (°C)
t	$T(t)$
1	36.37
3	36.23
5	36.21
7	36.26
8	36.38
9	36.49
10	36.60
11	36.63
12	36.66
13	36.68
14	36.69
15	36.73
16	36.74
17	36.78
18	36.82
19	36.84
20	36.87
21	36.86
22	36.77
23	36.59

Interpolate the data with a periodic cubic spline and plot your solution for a two-day-period.

Solution:

设样条 $s(x)$ 在区间 $[a, b]$ 上, 型值点 (x_i, y_i) ($i = 0, \dots, n$) 的横坐标为 $a = x_0 < x_1 < \dots < x_n = b$
 假设 $s(x)$ 在相邻型值点之间为三次多项式, 即 $s(x)$ 在区间 $[x_i, x_{i+1}]$ 上的表达式为:

$$s_i(x) := a_i + b_i x + c_i x^2 + d_i x^3 \quad (i = 0, \dots, n-1)$$

因此 $s(x)$ 在整个 $[a, b]$ 区间中有 $4n$ 个待定系数.

它要满足以下条件:

- ① 通过型值点: (共 $2n$ 个方程)

$$\begin{cases} s_i(x_i) = y_i \\ s_i(x_{i+1}) = y_{i+1} \end{cases} \quad (i = 0, \dots, n-1)$$

- ② 在型值点处的一阶导数连续: (共 $n-1$ 个方程)

$$s'_i(x_{i+1}) = s'_{i+1}(x_{i+1}) \quad (i = 0, \dots, n-2)$$

- ③ 在型值点处的二阶导数连续: (共 $n-1$ 个方程)

$$s''_i(x_{i+1}) = s''_{i+1}(x_{i+1}) \quad (i = 0, \dots, n-2)$$

- ④ 周期样条 (Periodic Spline) 边界条件: (共 2 个方程)

假设样条的周期为 $b - a$, 需要要求边界点 $(x_0 + b - a)$ 和 x_n 处的衔接是光滑的: (例如心电图)

$$\begin{aligned} s'_0(x_0) &= s'_{n-1}(x_n) \\ s''_0(x_0) &= s''_{n-1}(x_n) \end{aligned}$$

求解 $4n$ 阶线性方程组的代价 $O(n^3)$ 还是太大了.

我们需要想办法将 $4n$ 阶线性方程组变为低阶线性方程组.

注意到我们有 $n + 1$ 个导数值 $s'(x_0), \dots, s'(x_n)$ 未知.

我们可以利用它们进行三次 Hermite 插值.

定义:

$$\begin{aligned} k_i &:= s'(x_i) & (i = 0, 1, \dots, n) \\ h_i &:= x_{i+1} - x_i & (i = 0, 1, \dots, n) \\ \phi(x) &:= 3x^2 - 2x^3 \\ \varphi(x) &:= x^3 - x^2 \end{aligned}$$

其中 $k_0, k_1, \dots, k_{n-1}, k_n$ 都是未知的.

于是根据插值条件 $\begin{cases} s_i(x_i) = y_i \\ s_i(x_{i+1}) = y_{i+1} \\ s'_i(x_i) = k_i \\ s'_i(x_{i+1}) = k_{i+1} \end{cases}$ 和三次 Hermite 插值的结论可得 $s_i(x)$ 的表达式为:

$$\begin{aligned} s_i(x) &= y_i \phi\left(\frac{x_{i+1} - x}{h_i}\right) + y_{i+1} \phi\left(\frac{x - x_i}{h_i}\right) - k_i h_i \varphi\left(\frac{x_{i+1} - x}{h_i}\right) + k_{i+1} h_i \varphi\left(\frac{x - x_i}{h_i}\right) \\ s'_i(x) &= -\frac{y_i}{h_i} \phi'\left(\frac{x_{i+1} - x}{h_i}\right) + \frac{y_{i+1}}{h_i} \phi'\left(\frac{x - x_i}{h_i}\right) + k_i \varphi'\left(\frac{x_{i+1} - x}{h_i}\right) + k_{i+1} \varphi'\left(\frac{x - x_i}{h_i}\right) \\ s''_i(x) &= \frac{y_i}{h_i^2} \phi''\left(\frac{x_{i+1} - x}{h_i}\right) + \frac{y_{i+1}}{h_i^2} \phi''\left(\frac{x - x_i}{h_i}\right) - \frac{k_i}{h_i} \varphi''\left(\frac{x_{i+1} - x}{h_i}\right) + \frac{k_{i+1}}{h_i} \varphi''\left(\frac{x - x_i}{h_i}\right) \\ s'''_i(x) &= -\frac{y_i}{h_i^3} \phi'''\left(\frac{x_{i+1} - x}{h_i}\right) + \frac{y_{i+1}}{h_i^3} \phi'''\left(\frac{x - x_i}{h_i}\right) + \frac{k_i}{h_i^2} \varphi'''\left(\frac{x_{i+1} - x}{h_i}\right) + \frac{k_{i+1}}{h_i^2} \varphi'''\left(\frac{x - x_i}{h_i}\right) \\ &\equiv 12 \frac{y_i - y_{i+1}}{h_i^3} + 6 \frac{k_i + k_{i+1}}{h_i^2} & (i = 0, 1, \dots, n-1) \\ &= -4\beta_i \eta_i + 6\beta_i^2(k_i + k_{i+1}) \end{aligned}$$

where $\phi(x) = 3x^2 - 2x^3$, $\varphi(x) = x^3 - x^2$

$$\phi'(x) = 6x - 6x^2, \quad \varphi'(x) = 3x^2 - 2x$$

$$\phi''(x) = 6 - 12x, \quad \varphi''(x) = 6x - 2$$

$$\phi'''(x) \equiv -12, \quad \varphi'''(x) \equiv 6$$

$$\beta_i = \frac{1}{h_i}, \quad \eta_i = 3 \frac{y_{i+1} - y_i}{h_i^2}$$

现考虑二阶导的插值条件 $s''_i(x_{i+1}) = s''_{i+1}(x_{i+1})$:

$$\begin{aligned}
s_i''(x_{i+1}) &= \frac{y_i}{h_i^2} \phi''(0) + \frac{y_{i+1}}{h_i^2} \phi''(1) - \frac{k_i}{h_i} \varphi''(0) + \frac{k_{i+1}}{h_i} \varphi''(1) \\
&= 6 \frac{y_i}{h_i^2} - 6 \frac{y_{i+1}}{h_i^2} + 2 \frac{k_i}{h_i} + 4 \frac{k_{i+1}}{h_i} \\
s_{i+1}''(x_{i+1}) &= \frac{y_{i+1}}{h_{i+1}^2} \phi''(1) + \frac{y_{i+2}}{h_{i+1}^2} \phi''(0) - \frac{k_{i+1}}{h_{i+1}} \varphi''(1) + \frac{k_{i+2}}{h_{i+1}} \varphi''(0) \\
&= -6 \frac{y_{i+1}}{h_{i+1}^2} + 6 \frac{y_{i+2}}{h_{i+1}^2} - 4 \frac{k_{i+1}}{h_{i+1}} - 2 \frac{k_{i+2}}{h_{i+1}}
\end{aligned}$$

$$\begin{aligned}
s_i''(x_{i+1}) &= s_{i+1}''(x_{i+1}) \\
&\Downarrow \\
6 \frac{y_i}{h_i^2} - 6 \frac{y_{i+1}}{h_i^2} + 2 \frac{k_i}{h_i} + 4 \frac{k_{i+1}}{h_i} &= -6 \frac{y_{i+1}}{h_{i+1}^2} + 6 \frac{y_{i+2}}{h_{i+1}^2} - 4 \frac{k_{i+1}}{h_{i+1}} - 2 \frac{k_{i+2}}{h_{i+1}} \\
&\Downarrow \\
\beta_i k_i + \alpha_{i+1} k_{i+1} + \beta_{i+1} k_{i+2} &= \eta_i + \eta_{i+1}
\end{aligned}$$

where
$$\begin{cases}
h_i = x_{i+1} - x_i & (i = 0, \dots, n-1) \\
\beta_i = \frac{1}{h_i} & (i = 0, \dots, n-1) \\
\alpha_{i+1} = 2(\beta_i + \beta_{i+1}) & (i = 0, \dots, n-2) \\
\eta_i = 3 \frac{y_{i+1} - y_i}{h_i^2} & (i = 0, \dots, n-1)
\end{cases}$$

现考虑周期样条边界条件:

假设样条的周期为 $b - a = x_n - x_0$, 要求边界点 x_0 和 x_n 处的衔接是光滑的:

$$\begin{aligned}
s_0'(x_0) &= s_{n-1}'(x_n) \\
s_0''(x_0) &= s_{n-1}''(x_n) \\
&\Downarrow \\
k_0 = s_0'(x_0) &= s_{n-1}'(x_n) = k_n \\
2(\eta_0 - 2\beta_0 k_0 - \beta_0 k_1) &= s_0''(x_0) = s_{n-1}''(x_n) = 2(-\eta_{n-1} + \beta_{n-1} k_{n-1} + 2\beta_{n-1} k_n) \\
&\Downarrow \\
2(\beta_0 + \beta_{n-1}) k_0 + \beta_0 k_1 + \beta_{n-1} k_{n-1} &= \eta_{n-1} + \eta_0
\end{aligned}$$

若额外定义 $\alpha_0 = 2(\beta_0 + \beta_{n-1})$, 则我们有 $\alpha_0 k_0 + \beta_0 k_1 + \beta_{n-1} k_{n-1} = \eta_{n-1} + \eta_0$

此外, 在 $\beta_{n-2} k_{n-2} + \alpha_{n-1} k_{n-1} + \beta_{n-1} k_n = \eta_{n-2} + \eta_{n-1}$ 中代入 $k_n = k_0$

即可得到 $\beta_{n-1} k_0 + \beta_{n-2} k_{n-2} + \alpha_{n-1} k_{n-1} = \eta_{n-2} + \eta_{n-1}$

于是方程组为:

$$\begin{cases}
\alpha_0 k_0 + \beta_0 k_1 + \beta_{n-1} k_{n-1} = \eta_{n-1} + \eta_0 & (\text{additional condition}) \\
\beta_i k_i + \alpha_{i+1} k_{i+1} + \beta_{i+1} k_{i+2} = \eta_i + \eta_{i+1} & (i = 0, \dots, n-2) \\
\beta_{n-1} k_0 + \beta_{n-2} k_{n-2} + \alpha_{n-1} k_{n-1} = \eta_{n-2} + \eta_{n-1} & (\text{additional condition})
\end{cases}$$

$$\Downarrow$$

$$\begin{bmatrix}
\alpha_0 & \beta_0 & & & \beta_{n-1} \\
\beta_0 & \alpha_1 & \beta_1 & & \\
& \beta_1 & \alpha_2 & \beta_2 & \\
& & \ddots & \ddots & \ddots \\
& & & \beta_{n-3} & \alpha_{n-2} & \beta_{n-2} \\
\beta_{n-1} & & & \beta_{n-2} & \alpha_{n-1}
\end{bmatrix}
\begin{bmatrix}
k_0 \\
k_1 \\
k_2 \\
\vdots \\
k_{n-2} \\
k_{n-1}
\end{bmatrix}
=
\begin{bmatrix}
\eta_{n-1} + \eta_0 \\
\eta_0 + \eta_1 \\
\eta_1 + \eta_2 \\
\vdots \\
\eta_{n-3} + \eta_{n-2} \\
\eta_{n-2} + \eta_{n-1}
\end{bmatrix}$$

这是一个严格对角占优的 n 阶循环三对角阵, 使用 Gauss 消去法求解的代价为 $O(n)$

严格对角占优意味着 Gauss 消去法不用选主元
于是循环三对角矩阵的 LU 分解为:

$$\begin{bmatrix} x & x & & & x \\ x & x & x & & \\ & x & x & x & \\ & & x & x & x \\ x & & & x & x \end{bmatrix} = \begin{bmatrix} x & & & & \\ x & x & & & \\ & x & x & & \\ & & x & x & \\ x & x & x & x & x \end{bmatrix} \begin{bmatrix} x & x & & & x \\ & x & x & & x \\ & & x & x & x \\ & & & x & x \\ & & & & x \end{bmatrix}$$

每列只需两次行消元, $O(1)$ 级别代价

Gauss 消元法总代价 $O(n)$

回代法和前代法各自代价为 $O(n)$

因此求解严格对角占优的 n 阶循环三对角阵的代价为 $O(n)$

实现三次周期样条插值的函数:

```
% Helper functions
phi = @(x) 3 * x.^2 - 2 * x.^3;
varphi = @(x) x.^3 - x.^2;

% Sample data
nodes = [1, 36.37;
         3, 36.23;
         5, 36.21;
         7, 36.26;
         8, 36.38;
         9, 36.49;
         10, 36.60;
         11, 36.63;
         12, 36.66;
         13, 36.68;
         14, 36.69;
         15, 36.73;
         16, 36.74;
         17, 36.78;
         18, 36.82;
         19, 36.84;
         20, 36.87;
         21, 36.86;
         22, 36.77;
         23, 36.59;
         25, 36.37]; % 最后一个点与第一个点相隔一个完整周期 (24h)

x_nodes = nodes(:,1)';
y_nodes = nodes(:,2)';
n = length(x_nodes); % 节点数

% 构造循环三对角系统
h = x_nodes(2:end) - x_nodes(1:end-1);
beta = 1 ./ h;
alpha = 2*[beta(1) + beta(end), beta(1:end-1) + beta(2:end)];
A = diag(alpha) + ...
    diag(beta(1:end-1), -1) + ...
    diag(beta(1:end-1), 1);
```

```

A(1, end) = beta(end);
A(end, 1) = beta(end);

% 构造右端向量
eta = 3 * (y_nodes(2:end) - y_nodes(1:end-1)) ./ (h.^2);
b = [eta(1) + eta(end), eta(1:end-1) + eta(2:end)]; % 注意它是行向量

% 求解导数
k = A \ b'; % 注意此时 k 是列向量，需要转为行向量
k = k';
k = [k, k(1)]; % 在末端补充导数

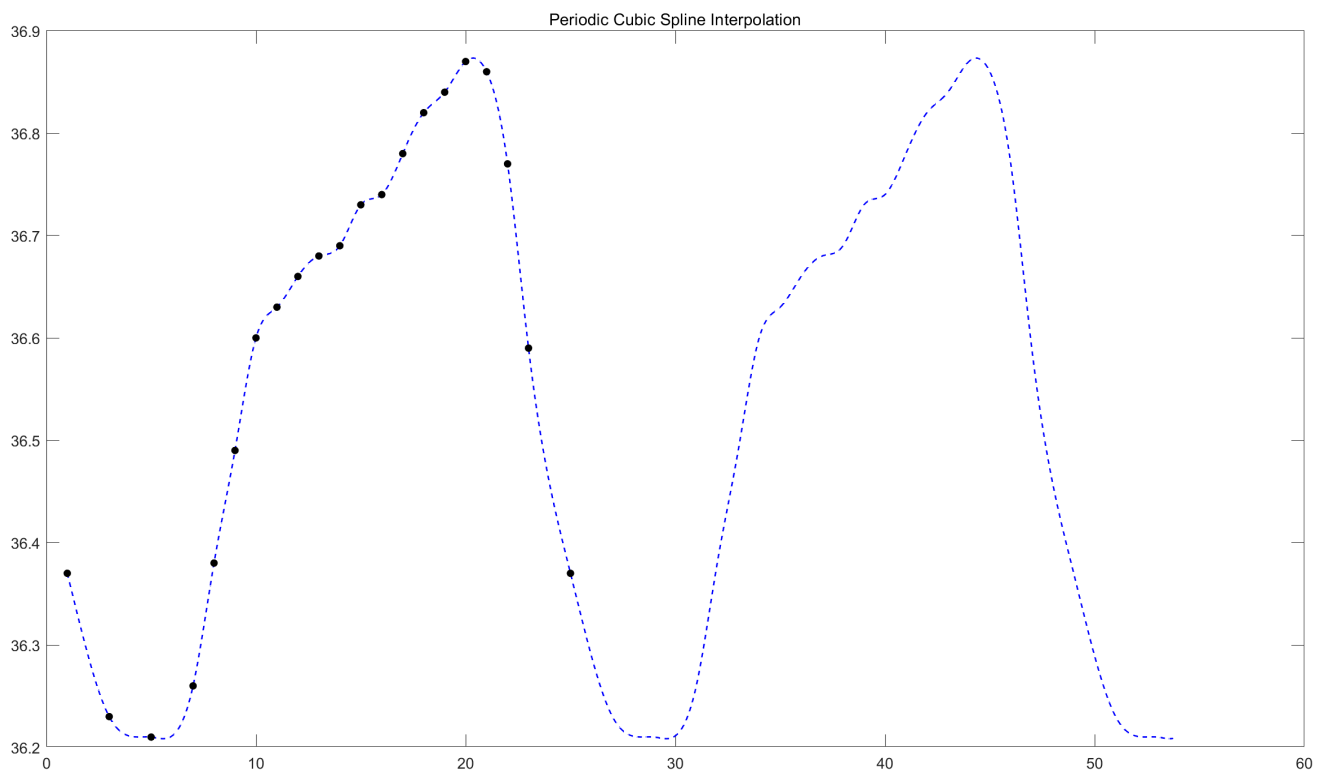
% 段序号
period = x_nodes(end) - x_nodes(1);
x_fine = linspace(x_nodes(1), x_nodes(1) + 2.2*period, 1000);
x_adj = mod(x_fine - x_nodes(1), period) + x_nodes(1);
index = discretize(x_adj, x_nodes);
index(end) = n-1;

% 分段计算样条值
y_interp = y_nodes(index) .* phi((x_nodes(index + 1) - x_adj) ./ h(index)) + ...
           y_nodes(index + 1) .* phi((x_adj - x_nodes(index)) ./ h(index)) - ...
           h(index) .* k(index) .* varphi((x_nodes(index + 1) - x_adj) ./ h(index)) + ...
           h(index) .* k(index + 1) .* varphi((x_adj - x_nodes(index)) ./ h(index));

% 绘图
figure; % 存疑：我发现 y_interp 的最后一项会非常大（待debug）
plot(x_fine(1:end-1), y_interp(1:end-1), 'b--', 'Linewidth', 1); hold on;
scatter(x_nodes, y_nodes, 25, 'k', 'filled');
title('Periodic Cubic Spline Interpolation');

```

运行结果:



Problem 6 (optional)

2. Sometimes we are interested in finding a curve that (approximately) passes through the given data points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. The curve is not necessarily of the form $y = f(x)$ because a straight line parallel to the y -axis may intersect with the curve at multiple points. One strategy is to perform two cubic spline interpolations (or cubic spline fittings) $x = x(t)$ and $y = y(t)$ by choosing an appropriate sequences of t_i 's. According to differential geometry, the best parameterization is to choose t as the arc length. In our case, we can replace arc length by straight line distance since we only have a discrete data set. Use this strategy to interpolate the following data sets and visualize your results.

(1) A smooth curve that connects (in turn)

$$(1, 1), (0, 2), (-1, 1), (0, 0), (1, -1), (0, -2), (-1, -1).$$

(2) A smooth closed curve that connects (in turn)

$$(3, 0), (2, 2), (0, 3), (-2, 2), (-3, 0), (-2, -2), (0, -3), (2, -2).$$

Problem 7 (optional)

Launch an image processing program (e.g., `mspaint` on Windows).

Open your left hand naturally, and put it on the computer screen.

Then use the mouse to sketch the outline of your hand (A few discrete points would suffice).

Reconstruct the outline of your hand with (piecewise) algebraic curves and visualize the result.

- **Hint:** 使用 Problem 6 的方法，其中弧长用点的直线距离近似。

Problem 8 (optional)

Let $x_0, x_1, \dots, x_n$ be distinct real numbers.

For any sufficiently smooth function f , show that:

$$f[x_0, x_1, \dots, x_n] = \iint \cdots \int_{\mathcal{T}_n} f^{(n)}(t_0 x_0 + t_1 x_1 + t_2 x_2 + \cdots + t_n x_n) dt_1 dt_2 \cdots dt_n$$

$$\text{where } \mathcal{T}_n := \{(t_1, t_2, \dots, t_n) : t_i \geq 0 \text{ for all } i = 0, 1, \dots, n \text{ and } t_0 + t_1 + \cdots + t_n = 1\}$$

Problem 9 (optional)

Try to simplify Newton's interpolation polynomial for equally spaced interpolation nodes:

$$x_0 < x_1 < \cdots < x_n$$

$$\text{where } x_i = x_0 + ih \text{ for } i = 0, 1, \dots, n$$

- **Hint:** 参见 FDU 数值算法 2. 插值与拟合 2.2.1 等距节点插值

Problem 10 (optional)

- **Hint:** 参见 FDU 数值算法 2. 插值与拟合 2.3.2 Hermite 插值

Hermite 插值差商表示例:

x_1	x_1	x_1	x_2	x_2
$f(x_1)$	$f(x_1)$	$f(x_1)$	$f(x_2)$	$f(x_2)$
	$f'(x_1)/1!$	$f'(x_1)/1!$	$f[x_1, x_2]$	$f'(x_2)/1!$
		$f''(x_1)/2!$	$f[x_1, x_1, x_2]$	$f[x_1, x_2, x_2]$
			$f[x_1, x_1, x_1, x_2]$	$f[x_1, x_1, x_2, x_2]$
				$f[x_1, x_1, x_1, x_2, x_2]$

1. Find the polynomial $p(x)$ with lowest degree that satisfies $f(1) = f'(1) =$ _____
 $f''(1) = 3$, $f(2) = f'(2) = f''(2) = 1$, and $f(3) = f'(3) = 2$.

解: 做对应的差商表如下:

x	1	1	1	2	2	2	3	3
y	3	3	3	1	1	1	2	2
		3	3	-2	1	1	1	2
			3	-5	3	1	0	1
				-8	8	-2	-1	1
					16	-10	$\frac{1}{2}$	2
						-26	$\frac{21}{4}$	$\frac{3}{4}$
							$\frac{125}{8}$	$-\frac{9}{4}$
								$-\frac{143}{16}$

(注: Hermite 插值多项式的系数也可以由对应的
 exercise-1-coef.m 文件自动生成)

$$\Rightarrow p(x) = 3 + 3(x-1) + 3(x-1)^2 - 8(x-1)^3 + 16(x-1)^3(x-2) - 26(x-1)^3(x-2)^2 + \frac{125}{8}(x-1)^3(x-2)^3 - \frac{143}{16}(x-1)^3(x-2)^3(x-3)$$

Problem 11 (optional)

Approximate the sine function over the closed interval $[0, 2\pi]$ using **piecewise cubic Hermite interpolation**, and visualize your result.

You are recommended to partition the interval with n equally spaced interpolation nodes for $n = 2, 3, 5, 9$.

- **Hint:** 参见 FDU 数值算法 2. 插值与拟合 2.3.2 Hermite 插值